

EMBEDDED SYSTEMS • MICROCONTROLLER ARCHITECTURE

STM32 Low-Power Modes and Wake-Up Mechanisms

A technical study of low-power architecture, register-level control, and interrupt-driven wake-up on the STM32L4 ultra-low-power microcontroller family

Selected Device: STM32L476RG

Core: Arm Cortex-M4F

ASSIGNMENT ROADMAP

From Datasheet to Working Low-Power Circuit

The objective of this project is to demonstrate the full engineering path used when adopting a new microcontroller: reading manufacturer documentation, understanding the register model, and translating that understanding into a real low-power firmware design — validated by simulation.



Why STM32?

32-bit Cortex-M core, rich peripheral set, and a dedicated ultra-low-power (L4) product line built for battery-operated and energy-harvested applications — a strong contrast case against 8-bit AVR devices such as the ATmega328P.

Method

All architectural claims in this presentation are derived from ST Microelectronics' official Datasheet and Reference Manual for the selected device, cross-checked against the Cortex-M4 Generic User Guide for core-level registers.

Deliverable

A register-accurate explanation of every STM32 low-power mode, the exact bits that control them, the sources that can wake the device, and a final head-to-head comparison with the ATmega328P.

PART 1 · SELECTED DEVICE

STM32L476RG — Ultra-Low-Power Cortex-M4 MCU

- **Family:** STM32L4 — "Ultra-Low-Power" series
- **Core:** Arm Cortex-M4 with FPU, up to 80 MHz
- **Flash / SRAM:** 1 MB Flash, 128 KB SRAM (96 KB SRAM1 + 32 KB SRAM2)
- **Package used in labs:** LQFP64 (Nucleo-L476RG board)
- **Power figures (typical):** Run $\approx 100 \mu\text{A}/\text{MHz}$, Stop 2 $\approx 1.1 \mu\text{A}$, Standby $\approx 0.4 \mu\text{A}$, Shutdown $\approx 30 \text{ nA}$ (values from datasheet electrical characteristics, $V_{\text{DD}} = 3.0 \text{ V}$, room temp.)
- **Why chosen:** Exposes every low-power mode named in the assignment (Sleep $\rightarrow$ Shutdown), making it ideal for a complete comparative study.

Primary Sources

Datasheet: DS10198 — "STM32L476xx: Arm Cortex-M4 32-bit MCU" (ST Microelectronics)

Reference Manual: RM0351 — "STM32L47xxx/L48xxx advanced Arm-based 32-bit MCUs" (PWR, RCC, EXTI, GPIO chapters)

Core reference: Arm Cortex-M4 Processor Technical/Generic User Guide (SCB, NVIC, SCR)

Board reference: UM1724 — Nucleo-64 boards User Manual

Document map

Datasheet $\rightarrow$ electrical limits & pinout | Reference Manual $\rightarrow$ register definitions (this deck) | Programming Manual PM0214 $\rightarrow$ Cortex-M4 instruction set (WFI/WFE)

System Architecture Overview

Cortex-M4 Core

32-bit RISC core, 3-stage pipeline, single-cycle MAC, optional FPU/DSP extensions. Executes from Flash via I-Code/D-Code buses (ICode, DCode, System bus — Arm AHB-Lite).

Bus Matrix

Multi-layer AHB bus matrix connects the core, DMA controllers, and Flash/SRAM interfaces to bridges leading to APB1 and APB2 peripheral buses.

Power Domains

VDD (I/O + digital), VDDA (analog), VBAT (RTC + backup registers), and an internal LDO that supplies the core logic — each domain can be shut down independently in low-power modes.

Cortex-M4 Core + NVIC + FPU

ICode / DCode / System bus



AHB Bus Matrix

AHB1 · AHB2 · AHB3



Flash / SRAM

1MB / 128KB

DMA1 / DMA2

Data movement

GPIO A-H

I/O ports



APB1 (Low-Speed) / APB2 (High-Speed)

Timers · USART · SPI/I2C · ADC · RTC · PWR/RCC ctrl

The Arm Cortex-M4 Core

- **ISA:** Thumb-2, 32-bit RISC pipeline (3-stage, single-issue)
- **FPU:** single-precision hardware floating point (Cortex-M4F)
- **DSP extensions:** single-cycle MAC, SIMD instructions
- **NVIC:** tightly coupled to the core for low-latency interrupts (see slide 8)
- **Sleep support built into the core:** the core itself implements `WFI`, `WFE` and the `SLEEPDEEP` bit that STM32 low-power modes rely on
- **Memory Protection Unit (MPU):** optional region-based protection, independent of power state

Core ↔ Power-mode relationship

The Cortex-M core does not itself define "Stop" or "Standby" — these are STM32-specific power states built by the PWR peripheral *on top of* the core's generic sleep behaviour. Setting `SCB->SCR.SLEEPDEEP` before executing `WFI` / `WFE` tells the STM32 power controller to enter one of its deep low-power modes instead of plain Cortex-M Sleep.

Clock System (RCC)

- **HSI16** — internal 16 MHz RC oscillator
- **MSI** — Multi-Speed Internal RC, 100 kHz–48 MHz, auto-trimmed by LSE; used as the default low-power system clock
- **HSE** — external crystal, 4–48 MHz
- **PLL** — generates high-speed SYSCLK (up to 80 MHz) from HSI16/MSI/HSE
- **LSI** — internal 32 kHz, drives IWDG
- **LSE** — external 32.768 kHz crystal, drives the RTC in all low-power modes

Individual peripheral clocks (AHB/APB) can be gated on/off in `RCC_AHBxENR` / `RCC_APBxENR` — the single largest lever for reducing active-mode current.



GPIO Architecture

- Up to 8 ports (**GPIOA...GPIOH**), each with 16 pins, independently clocked via `RCC_AHB2ENR`
- Each pin configurable as Input, Output, Alternate Function, or Analog via `GPIOx_MODER`
- `GPIOx_PUPDR` — pull-up/pull-down; `GPIOx_OTYPER` — push-pull/open-drain
- In **Standby** and **Shutdown**, GPIO state is lost unless configured through `PWR_PUCRx` / `PWR_PDCRx` (pull-up/down retention registers) — these hold a defined pin state while the I/O logic itself is powered down
- Selected pins double as **EXTI** lines and **WKUPx** wake-up pins — the main GPIO-based wake mechanism

Power-mode behaviour

Sleep / Stop 0-1-2: GPIO configuration and output states are fully retained (peripheral logic stays powered).

Standby / Shutdown: GPIO registers reset; only pins explicitly held by `PWR_PUCRx/PDCRx` (Standby) keep a defined level. On wake, the MCU restarts and GPIO must be reconfigured in software.

Interrupt Controller (NVIC + EXTI)

- **NVIC** (Nested Vectored Interrupt Controller) — part of the Cortex-M4 core; STM32L476 exposes up to 82 maskable interrupt channels with 16 programmable priority levels
- Handles nesting, tail-chaining, and low-latency exception entry (fixed ~12-cycle latency)
- **EXTI** (Extended Interrupt/Event controller) — a separate STM32 peripheral that routes up to 16 GPIO lines plus internal signals (RTC, PVD, USB, COMP...) into NVIC interrupt lines *or* into core wake-up events
- EXTI is the key bridge that lets a GPIO edge wake the core even while most clocks are stopped

Signal path for a wake-up

GPIO pin edge → EXTI line (masked/triggered) → sets EXTI_PR

pending bit → (a) routed to NVIC as an interrupt if unmasked, and/or (b) recognised directly as a wake-up event by the core, independent of NVIC masking.

Peripheral / Bus Architecture

AHB (Advanced High-perf. Bus)

AHB1: DMA, CRC, Flash/SRAM interface.

AHB2: GPIO ports, ADC, RNG.

AHB3: QUADSPI. High-throughput, high-speed peripherals.

APB1 (Low-speed)

TIM2-7, RTC/BKP interface, WWDG, IWDG, SPI2/3, USART2/3, I2C1-3, **PWR**, **LPTIM1/2**. Split into APB1L/APB1H sub-domains.

APB2 (High-speed)

TIM1/8, USART1, SPI1, ADC control, SYSCFG, **EXTI**, SDMMC. Runs at full PCLK2 for time-critical peripherals.

Design implication for low power

Because every peripheral is clock-gated independently (RCC enable bits), firmware can disable exactly the blocks it does not need before entering Sleep, and the STM32 power controller automatically gates entire bus domains in Stop/Standby/Shutdown — this fine granularity is the main structural reason STM32 achieves much lower deep-sleep current than simpler 8-bit MCUs.

Power-Control System (PWR)

- Dedicated **PWR** peripheral manages voltage regulation, voltage scaling, and all low-power mode transitions
- **Internal LDO regulator** with two ranges: Range 1 (high performance, up to 80 MHz) and Range 2 (low power, up to 26 MHz) — selected via `PWR_CR1.VOS`
- **Low-power regulator** automatically supplies core logic in Stop modes at reduced output
- **Power-Voltage Detector (PVD)** monitors VDD against a programmable threshold and can raise an interrupt on brown-out risk
- Independent **VBAT** domain keeps the RTC, LSE oscillator, and backup registers alive from a coin-cell even when VDD is completely removed

Domains PWR can shut down

- Cortex-M4 core & most SRAM
- Flash memory
- High-speed clocks (HSI16/HSE/PLL)
- Digital peripherals on APB/AHB
- I/O logic (Standby/Shutdown)

Always-on in every mode: VBAT domain (RTC, LSE, backup registers) and, in Standby, optionally one SRAM2 bank for data retention.

Low-Power Mode Comparison — STM32L476

Mode	CPU	Clocks	RAM / Registers	Peripherals	Typ. current	Wake latency
Sleep	Stopped	All clocks running	Fully retained	All active	~ mA range	Immediate
Low-power Run	Running (≤2 MHz)	MSI low freq.	Fully retained	Active, limited	~9 µA/MHz	n/a (running)
Low-power Sleep	Stopped	Low-freq clocks only	Fully retained	Active, limited	few µA	Fast
Stop 0	Stopped	HSI16/HSE/PLL off	Fully retained	LSE/LSI peripherals only	~3.5 µA	~5 µs
Stop 1	Stopped	Same as Stop 0	Fully retained	Reduced regulator	~1.1 µA	~6 µs
Stop 2	Stopped	Same as Stop 0/1	Fully retained	Further reduced set	~0.75 µA	~10 µs
Standby	Off (reset on wake)	Only LSE/LSI (RTC)	SRAM2 optional (8/32 KB); registers lost	RTC, WKUP pins, IWDG	~0.4 µA	~µs–ms range*
Shutdown	Off (full reset)	Only LSE optional	Lost (backup registers survive)	RTC (LSE), WKUP pins	~30 nA	slowest*

*Standby/Shutdown wake behaves like a power-on reset — the application re-boots from the reset vector; "wake time" includes reset + clock start-up. Figures are typical values from DS10198 electrical characteristics and vary with temperature and configuration.

Sleep & Low-Power Sleep Modes

Sleep mode

Only the Cortex-M4 clock (HCLK to the core) is stopped; all peripherals and all clock sources keep running. Entered with `WFI` or `WFE` while `SCR.SLEEPDEEP = 0`. Any enabled interrupt (or event, for WFE) wakes the core instantly with full context preserved — no reconfiguration needed.

Low-power Sleep mode

Same entry mechanism, but the device is first put into Low-power Run (regulator in low-power mode, clock ≤ 2 MHz) before Sleep is entered. This trades wake responsiveness/throughput for lower average current in applications that wake frequently but briefly.

When to use

Sleep: short idle gaps between bursts of activity where instant, full-speed resumption matters (e.g. waiting for a DMA transfer or ADC conversion to finish).

Low-power Sleep: systems polling a sensor at low, steady frequency where every microamp counts but Stop-mode wake latency is undesirable.

PART 2 · LOW-POWER MODES

Stop 0 / Stop 1 / Stop 2 Modes

- Entered by setting `SCB->SCR.SLEEPDEEP = 1` and setting `PWR_CR1.LPMS` to select Stop 0/1/2, then executing `WFI` / `WFE`
- All high-speed clocks (HSI16, HSE, PLL) are switched off; only LSI/LSE keep running for RTC and IWDG
- All SRAM and register contents are fully retained — the CPU resumes exactly where it left off (no reset)
- **Stop 0:** main regulator kept on → fastest wake-up (~5 µs), higher residual current
- **Stop 1:** main regulator in low-power mode → balanced current/wake-time trade-off (default choice for most designs)
- **Stop 2:** most peripherals and additional regulator stages powered down → lowest current of the three, but a smaller subset of peripherals remains usable as wake-up sources (limits: fewer analog blocks stay available)
- Wake sources: any EXTI line (GPIO, PVD, RTC alarm/wake-up timer/tamper), plus, in Stop 0/1 only, certain communication peripherals (LPUART, I2C3) configured for address-match wake-up

PART 2 · LOW-POWER MODES

Standby Mode

- Deepest mode that can still retain data: one or two SRAM2 banks may optionally be kept powered (`PWR_CR3.RRS`) for a small retained data footprint
- Core logic, most of Flash, and the main regulator are powered off entirely
- Entered with `PWR_CR1.LPMS = "011"` , `SLEEPDEEP=1` , then `WFI` / `WFE`
- Wake-up sources: **WKUP1-WKUP5** pins, RTC alarm / wake-up timer / tamper / timestamp, IWDG reset, NRST pin
- On wake, the MCU executes a full reset sequence and reboots from the reset vector — the wake source is discoverable via `PWR_SR1` / `PWR_SCR` flags

GPIO retention in Standby

Normally I/O states are lost; but the `PWR_PUCRx` / `PWR_PDCRx` registers let firmware force a defined pull-up or pull-down on selected pins throughout Standby, preventing floating inputs on external circuitry while the MCU itself is powered down.

Shutdown Mode (Deepest — STM32L4-specific)

- Lowest power state available on STM32L4 (~30 nA typical) — *not available on all STM32 families*, which is why the assignment lists it as optional
- The internal main and low-power regulators are both switched off; VCORE is fully unpowered
- SRAM and register contents are **lost**; only the VBAT/backup domain (backup registers, RTC if LSE selected) can survive
- Entered with `PWR_CR1.LPMS = "1xx"` (Shutdown), `SLEEPDEEP=1`, then `WFI` / `WFE`
- Wake-up sources are limited to: **WKUP pins**, **RTC** events (if LSE kept running), and the **NRST** pin — no IWDG wake in Shutdown
- Wake-up always produces a full power-on-style reset and re-initialization

Typical use case

Devices with a multi-year coin-cell budget that wake only a few times per day on an RTC alarm or a button press — e.g. asset trackers, environmental data loggers, or wireless sensor nodes that spend >99.9% of their life doing nothing.

Important Registers — PWR (Power Control)

PWR_CR1

`LPMS[2:0]` selects the low-power mode (Stop0/1/2, Standby, Shutdown) · `LPR` enables Low-power Run · `VOS[1:0]` selects voltage scaling range 1/2

PWR_CR2

`PVDE` enables the Power-Voltage Detector · `PVDFT/PVDRT` configure PVD trigger edges · `USV` enables USB supply valid monitoring

PWR_CR3

`EWUP1–EWUP5` enable individual wake-up pins · `RRS` keeps SRAM2 powered in Standby · `APC` applies pull configuration

PWR_CR4

`WP1–WP5` set the polarity (rising/falling) of each wake-up pin

PWR_SR1 / PWR_SR2

`WUFx` flags identify which WKUP pin caused the last wake · `SBF` = woke from Standby · `REGLPF` , `VOSF` report regulator status

PWR_SCR / PWR_PUCRx / PWR_PDCRx

`PWR_SCR` clears wake-up flags (`CWUFx` , `CSBF`) · `PWR_PUCRx` / `PWR_PDCRx` hold per-pin pull-up/down during Standby

Important Registers — RCC (Reset & Clock Control)

RCC_CR

`MSION/HSION/HSEON/PLLON` enable each clock source · matching `*RDY` flags confirm stabilisation before switch-over

RCC_CFGR

`SW[1:0]` selects SYSCLK source · `HPRE`, `PPRE1`, `PPRE2` set AHB/APB prescalers — lowering these reduces dynamic power directly

RCC_BDCR

`LSEON` enables the 32.768 kHz crystal · `RTCSEL[1:0]` chooses the RTC clock source · `RTCEN` gates the RTC clock — this register lives in the VBAT domain

RCC_AHB1/2/3ENR

Per-peripheral clock gating for DMA, Flash interface, GPIO ports, ADC, RNG — disabling unused peripherals here is the first line of defence against wasted active-mode current

RCC_APB1ENR1/2, APB2ENR

Clock-enable bits for timers, USART, I2C/SPI, **PWR**, and **LPTIM1/2** — LPTIM specifically is designed to keep ticking in Stop modes

RCC_CSR

`LSION` enables the 32 kHz internal RC (drives IWDG) · reset-cause flags (`SFTRSTF`, `IWDGRSTF`, etc.) help firmware detect a Standby/Shutdown wake vs. a normal reset

Important Registers — Cortex-M SCR & NVIC

SCB→SCR (System Control Register)

Address: 0xE000ED10 (core peripheral)

SLEEPONEXIT (bit 1) — automatically re-enter sleep after handling an interrupt (ideal for interrupt-driven idle loops)

SLEEPDEEP (bit 2) — 0 selects Cortex-M Sleep; 1 tells the STM32 PWR controller to use Stop/Standby/Shutdown as configured in

PWR_CR1.LPMS

SEVONPEND (bit 4) — makes any newly pending interrupt generate a wake-up event, so **WFE** wakes even when that interrupt is disabled in NVIC

NVIC registers

NVIC_ISER0-2 / **ICER0-2** — set/clear enable bits per interrupt line

NVIC_ISPR / **ICPR** — set/clear pending status

NVIC_IPR0... — 8-bit priority field per interrupt (STM32L4 implements 4 priority bits → 16 levels)

A peripheral's NVIC line must be enabled for it to trigger a wake-up *interrupt* via WFI; for WFE-based wake it only needs to become pending (with **SEVONPEND** set).

Important Registers — EXTI

EXTI_IMR1 / IMR2

Interrupt Mask Register — a set bit lets the corresponding line generate an NVIC interrupt request

EXTI_EMR1 / EMR2

Event Mask Register — a set bit lets the line generate a core *event* for `WFE`, without necessarily firing an NVIC interrupt

EXTI_RTSR1/2 & FTSR1/2

Rising/Falling Trigger Selection — configure which pin edge(s) activate each EXTI line

EXTI_SWIER1/2

Software Interrupt Event Register — lets firmware force a line pending for testing the wake-up path

EXTI_PR1 / PR2

Pending Register — a set bit shows the line triggered; must be cleared by writing '1' to it (write-1-to-clear) in the ISR/after wake, or the line stays latched

PART 4 · WAKE-UP SOURCES

Wake-Up Sources by Mode

Source	Sleep	Stop 0/1/2	Standby	Shutdown
EXTI (GPIO pin, any edge)	Yes	Yes	Only WKUPx pins	Only WKUPx pins
RTC alarm / wake-up timer / tamper	Yes	Yes	Yes	Yes (LSE only)
General-purpose timers (TIMx)	Yes	No (clock stopped)	No	No
LPTIM1/2 (low-power timer)	Yes	Yes (Stop 0/1/2)	No	No
IWDG (independent watchdog)	Yes	Yes	Yes (resets device)	No
USART/LPUART address match	Yes	Stop 0/1 only	No	No
NRST pin	Yes	Yes	Yes	Yes

General rule: the deeper the mode, the fewer clocks survive — so only peripherals that can run from LSE/LSI (RTC, LPTIM, IWDG) or that are purely edge-triggered (EXTI/WKUP pins) remain viable wake-up sources in Stop 2, Standby, and Shutdown.

WFI vs. WFE — Two Ways to Sleep

WFI — Wait For Interrupt

Cortex-M instruction that suspends execution until *any enabled interrupt* (with sufficient priority) occurs. On wake, the core services the interrupt normally (enters the ISR) before returning to the instruction after WFI.

Used when: the wake action should always be handled by a specific interrupt service routine — the typical, most common way STM32 low-power code enters Sleep/Stop/Standby (used by ST's HAL, e.g.

```
HAL_PWR_EnterSTOPMode(..., PWR_STOPENTRY_WFI).
```

WFE — Wait For Event

Suspends execution until an *event*: an explicit `SEV` instruction elsewhere, the exclusive-access monitor being cleared, or — if `SEVONPEND` is set — any interrupt becoming pending, even if that interrupt is disabled in the NVIC. Waking via WFE does **not** automatically execute an ISR.

Used when: synchronising multiple cores/tasks, implementing lock-free wait loops, or waking on a condition without wanting the overhead/side-effects of actually entering an interrupt handler.

Key difference in one line

WFI wakes on interrupts and runs the ISR; **WFE** wakes on events (which may just be a pending flag) and returns control to the next instruction without necessarily servicing anything — giving finer control at the cost of extra bookkeeping (the programmer must check pending/event flags manually).

Practical Applications of STM32 Low-Power Modes

Battery sensor nodes

Environmental/IoT sensors that sample every few seconds: Stop 2 between samples, LPTIM/RTC wake-up timer as the periodic trigger.

Wearables & keypads

Standby/Shutdown with WKUP-pin wake on button press; RTC keeps time-of-day across the sleep period.

Asset trackers

Shutdown for months of storage/transport, RTC alarm wakes the system periodically to log GPS position, then returns to Shutdown.

USB / always-listening devices

Sleep mode between USB transactions or UART address-match wake in Stop 0/1, keeping high responsiveness with modest savings.

Motor / actuator control

Low-power Run/Sleep during idle periods between control-loop bursts, preserving full peripheral state for instant resumption.

Data-logging systems

SRAM2-retained Standby holds buffered readings while the rest of the chip is powered down between logging events.

ATmega328P vs. STM32 — Conceptual Differences

ATmega328P (8-bit AVR)

Single power-control register (`SMCR`) selects among six fixed sleep modes. Very simple model: the whole chip is either "awake" or in one coarse-grained sleep state. No dedicated ultra-low-power domain, no independent RTC peripheral, no register-level voltage scaling.

STM32 (32-bit Cortex-M)

Layered model: Cortex-M generic Sleep/Deep-Sleep + a dedicated PWR peripheral that defines several STM32-specific sub-modes (Stop 0/1/2, Standby, Shutdown), each with independently configurable clock domains, voltage regulator ranges, SRAM retention, and a rich wake-up-source matrix (EXTI, RTC, LPTIM, comms address-match).

Net effect

STM32's granularity lets a designer trade wake latency for current almost continuously across 6+ steps (mA → nA), while the ATmega328P offers a smaller, coarser set of trade-offs — simpler to program, but with less headroom for extreme energy-harvested designs.

FINAL COMPARISON

ATmega328P vs. STM32L476RG — Summary Table

Feature	ATmega328P	STM32L476RG
Sleep modes	Idle, ADC Noise Reduction, Power-down, Power-save, Standby, Extended Standby (6 modes)	Sleep, Low-power Run, Low-power Sleep, Stop 0/1/2, Standby, Shutdown (8 modes)
Deepest low-power mode	Power-down (~0.1–0.36 µA typ.)	Shutdown (~30 nA typ.)
Sleep-control registers	Single register: <code>SMCR</code> (SM2:0 + SE)	Multiple: <code>PWR_CR1-4</code> , <code>RCC_CR/CFGR</code> , <code>SCB→SCR</code>
Wake-up sources	INT0/INT1, pin-change interrupts, WDT, TWI address match, Timer2 async	EXTI lines, WKUP pins, RTC (alarm/wake-up timer/tamper), LPTIM, IWDG, comms address-match
External interrupt wake-up	Yes — INT0/INT1 and PCINT groups	Yes — up to 16 configurable EXTI lines + 5 WKUP pins
Timer operation during sleep	Only Timer2 (asynchronous, external 32 kHz crystal) in Power-save; others halted	LPTIM runs in Stop 0/1/2 from LSE/LSI; general timers only in Sleep
RTC operation	No dedicated RTC — Timer2 async used as a substitute time base	Dedicated RTC peripheral, runs in every mode down to Shutdown (with LSE)
RAM retention	Retained in all sleep modes (SRAM stays powered while VCC applied)	Retained in Sleep→Stop2; optional partial retention in Standby; lost in Shutdown
Typical applications	Simple, low-cost battery devices; Arduino-class sensors and toys	Advanced battery/energy-harvested IoT nodes, wearables, asset trackers

CONCLUSION

Key Takeaways

- STM32's power architecture layers a generic Cortex-M sleep model with a dedicated PWR peripheral, giving 6–8 distinct low-power modes versus the ATmega328P's single-register, six-mode model.
- Every mode is defined by exactly which clocks, regulators, and memory banks stay powered — always traceable back to specific bits in `PWR_CR1-4`, `RCC_CR/CFGR`, and `SCB→SCR`.
- `WFI` and `WFE` are the two core instructions that actually trigger mode entry — the choice between them determines whether the wake-up is handled through a normal ISR or through the lighter-weight event mechanism.
- EXTI and RTC are the universal wake-up backbone that keeps working even in the deepest modes, making them the natural building blocks for any battery-powered STM32 design.

Sources: ST Microelectronics DS10198 (Datasheet) & RM0351 (Reference Manual), Arm Cortex-M4 Generic User Guide, UM1724 Nucleo-64 User Manual.